

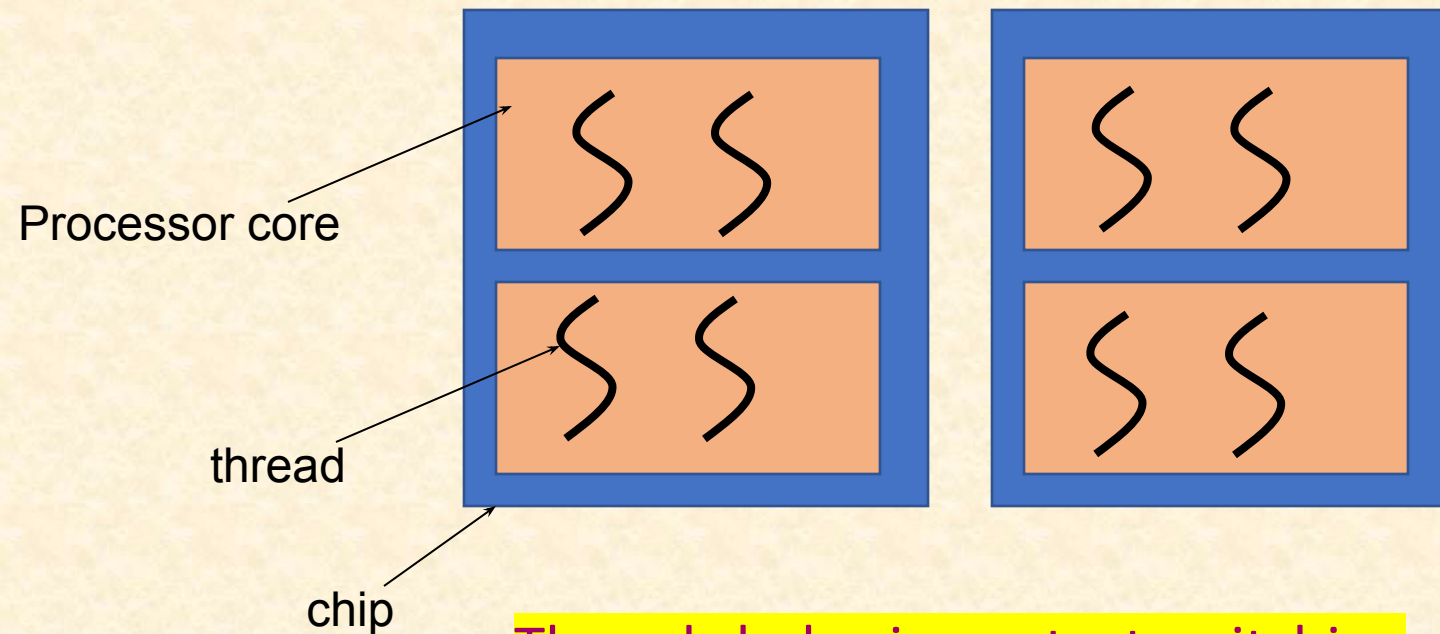
Operating Systems Overview: Part 2

Sk Subidh Ali

IIT Bhilai

Multiprocessors

- Multiple processors chips in a single system
 - Generally used in server computers
- Multiple cores in a single chip
- Multiple threads in a single core



Threads helps in context switching

Multiprocessors



Core i7 12700 12th Generation 2GHz 12 Cores 20 Threads

How many program can run simultaneously in primitive OS?

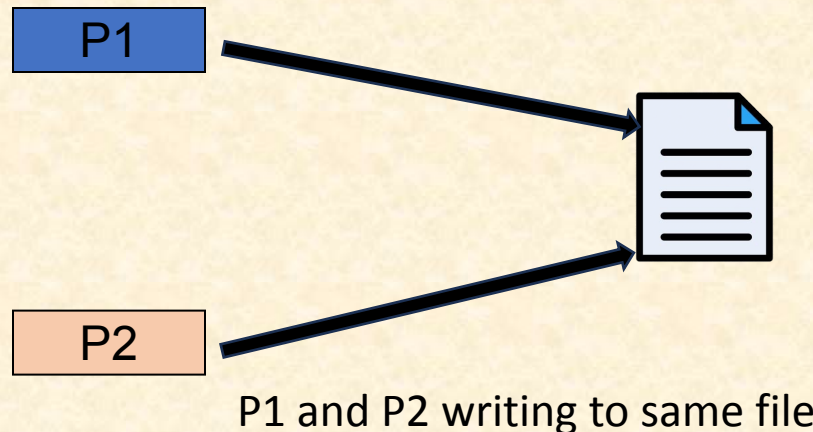
How many program can run simultaneously in multiprogramming OS?

How many program can run simultaneously in multiprogramming OS?

Challenges in multitasking systems

Challenge 1: Multiple program share common resource

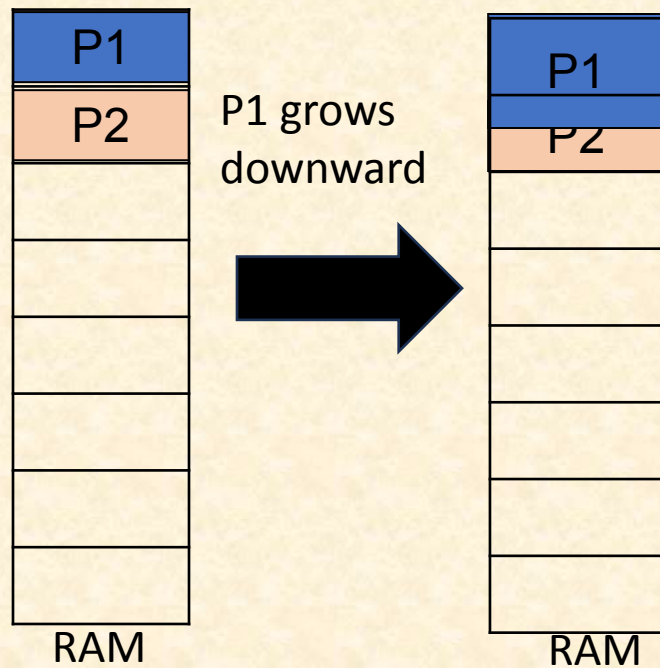
- Race condition: they may corrupt each others data



Synchronization: When P1 writing, P2 should not get access to the file in write mode, vice-versa.

Challenges in multitasking systems

Challenge 2: One program may corrupt another program by overwriting its memory



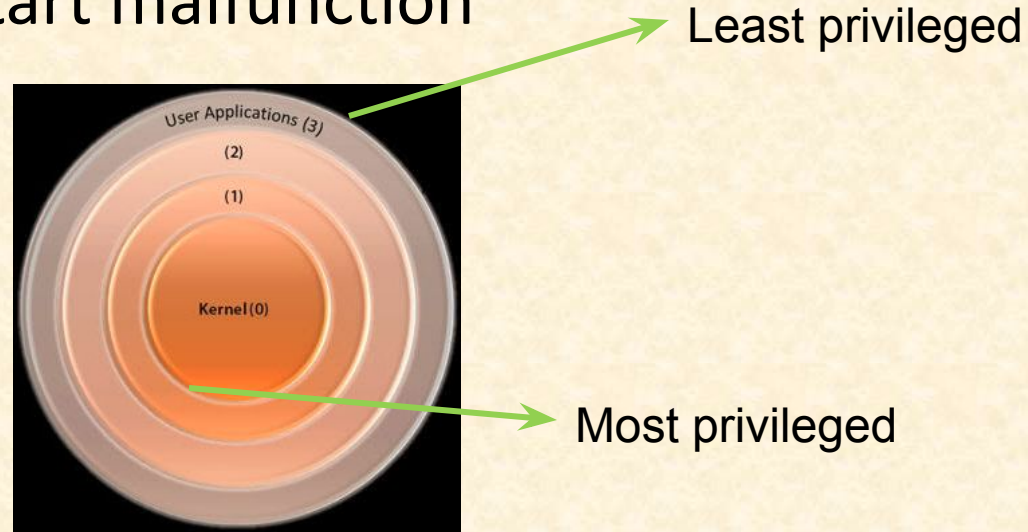
```
int main()
{
    int A[20],i=0;
    while(i>100){
        A[i]=i;
        i++;
    }
}
```

Isolation: Virtual memory isolates one process's memory from others

Challenges in multitasking systems

Challenge 3: A program may acquire resources of OS

- System will start malfunction



Privileged & unprivileged modes in CPUs :

- Applications unprivileged (user/unprivileged mode)
- OS privileged (privileged/supervisor mode)

OS and Security

- Why is it needed?

- Defend against internal or external attacks from viruses, worms, identity theft, theft of service.

- How is it achieved?

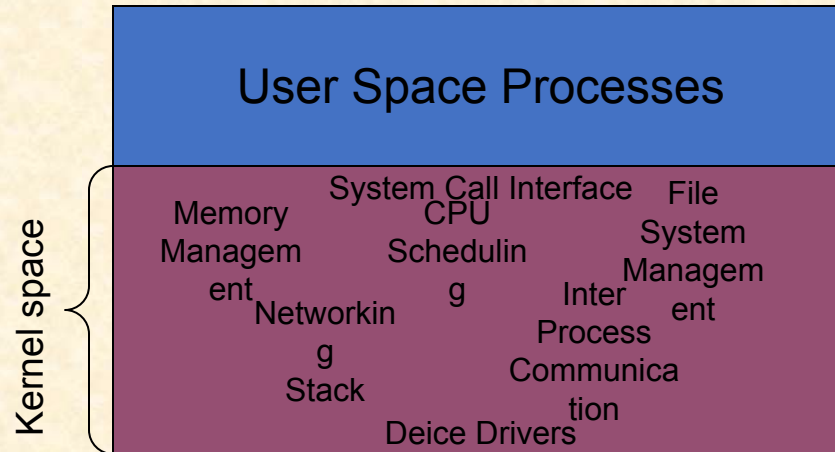
- Access Control
 - Passwords and Cryptography
 - Biometrics
 - Security assessment

File system

- Manages data persistency across reboot
- It allows a process to
 - Create, Open, read/write file, and delete files
- Provide abstract representation of secondary storage

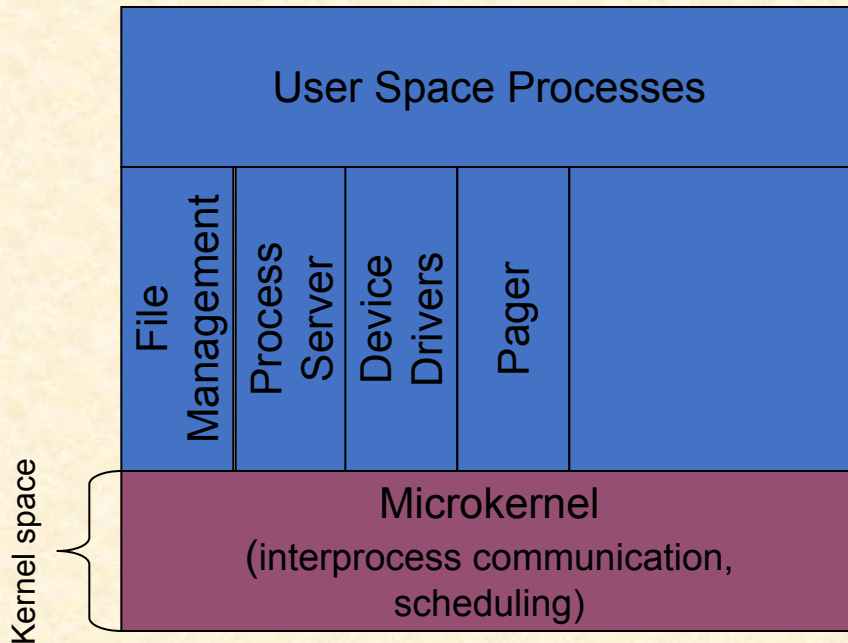
OS Structure :

Monolithic Structure



- Linux, MS-DOS, xv6
- All components of OS in kernel space
- **Cons** : Large size, difficult to maintain, likely to have more bugs, difficult to verify
- **Pros** : direct communication between modules in the kernel by procedure calls

OS Structure : Microkernel



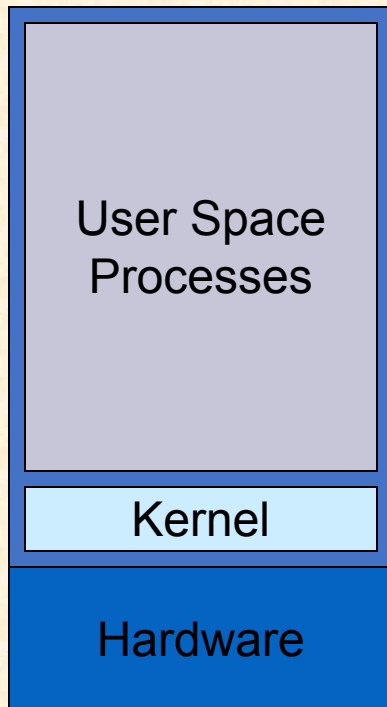
Eg. QNX and L4

- Highly modular.
 - Every component has its own space.
 - Interactions between components strictly through well defined interfaces (no backdoors)
- Kernel has basic inter process communication and scheduling
 - Everything else in user space.
 - Ideally kernel is so small that it fits the first level cache

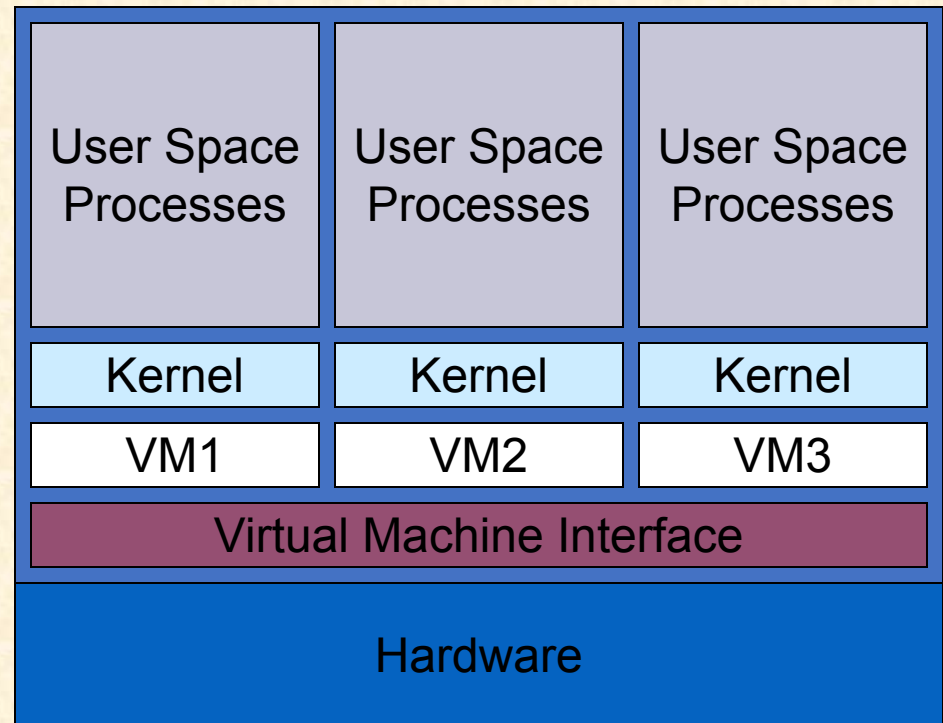
Monolithic vs Microkernels

	Monolithic	Microkernel
Inter process communication	Signals, sockets	Message queues
Memory management	Everything in kernel space (allocation strategies, page replacement algorithms,)	Memory management in user space, kernel controls only user rights
Stability	Kernel more 'crashable' because of large code size	Smaller code size ensures kernel crashes are less likely
I/O Communication (Interrupts)	By device drivers in kernel space. Request from hardware handled by interrupts in kernel	Requests from hardware converted to messages directed to user processes
Extendibility	Adding new features requires rebuilding the entire kernel	The micro kernel can be base of an embedded system or of a server
Speed	Fast (Less communication between modules)	Slow (Everything is a message)

Virtual Machines



No virtual Machines



With virtual Machines

Booting



Sk Subidh Ali

IIT Bhilai

Booting



CPU resets
IP=fix address(BIOS)



BIOS executed from
Firmware



BIOS executes MBR



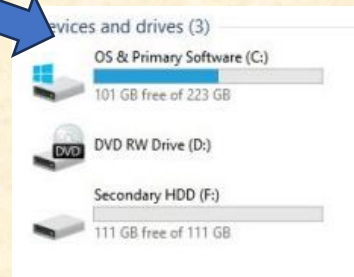
Load Kernel



Run init process



Bios /UEFI: Secure flash memory



Bios: Basic Input/Output System

Modern bios is a tiny OS in itself

Bios, once executed,

- Run POST (Power On Self Test)
- Runs MBR
- Boot hand off:
 - Bootloader executes

- Conduct quick test on RAM, graphics card, I/O, and other peripherals
- If test fails it throw error message

- It's the first 512 bytes of your primary partition
- It is created when you format a disc
- It contains partition table

MBR: Master Boot Record

Master boot record: first sector of boot device/disk

- **Partition table**: upto 4 partitions
 - Default: entire disk has one partition; multiple partitions: each can be a different volume/file system
- **Bootstrap code**: called stage 1 boot loader
 - First 440 bytes of 512 bytes
 - CPU loads bootstrap and starts executing it
 - It look for partition marked active; load code from that partition **boot sector**



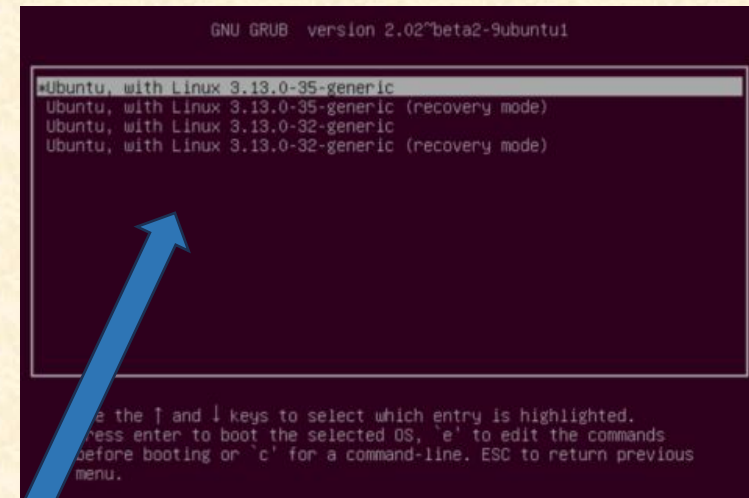
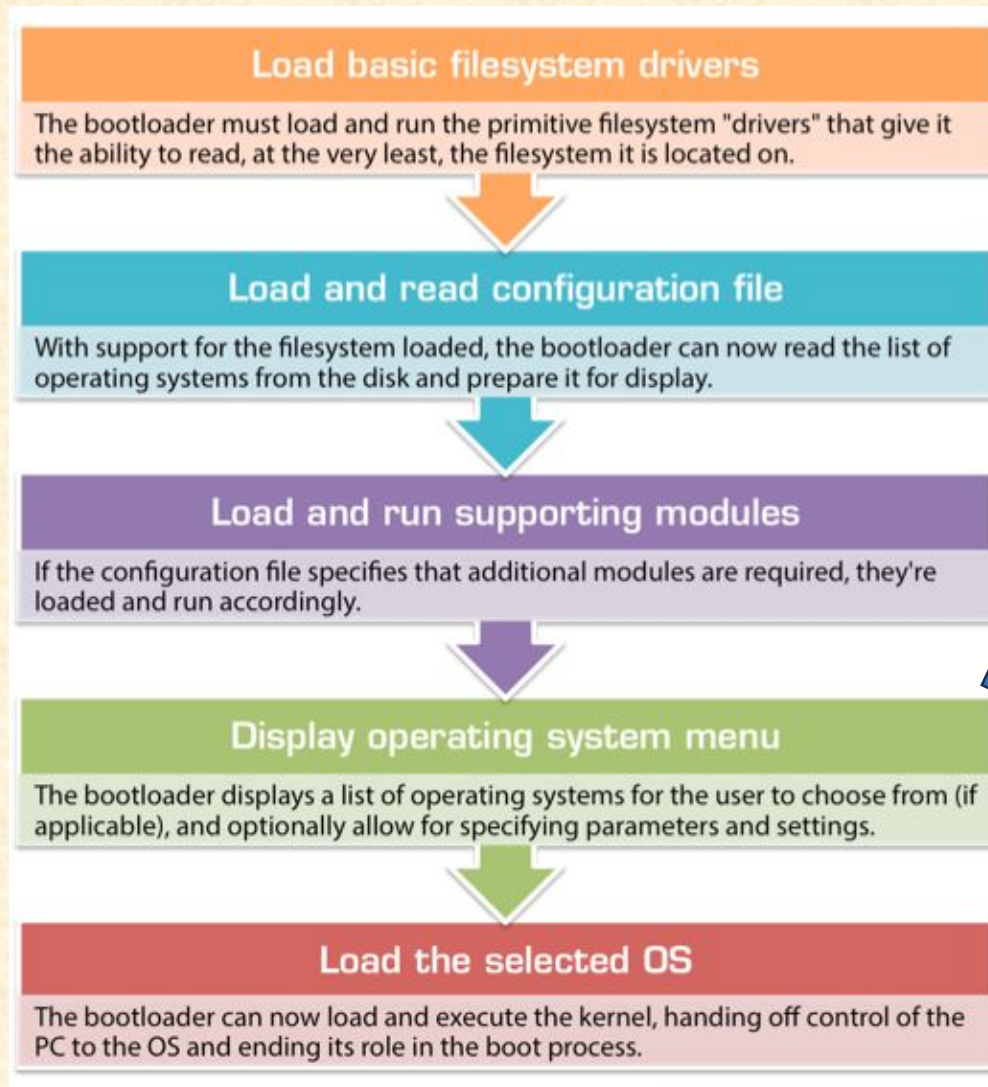
Partition Boot Sector

First 512 bytes of active partition

- First 3 bytes: JMP instruction; skip XX bytes to bootstrap code
 - Filesystem header: info specific to file system in volume
 - Bootstrap code: Second-stage boot loader
- Ends with JMP to next sector in partition for actual loader
 - Together, they look up a file stored on partition as a regular file (**Bootloader**) and tell CPU to execute the file
- Stored as a normal file on the file system



Bootloader



Kernel Initialization

- **Set up basic CPU state:** Kernel page tables, virtual memory, kernel stack.
- **Set up trap/interrupt infrastructure:** configuring **IDT (Interrupt Descriptor Table)**
- **Initialize memory management** — set up the physical memory allocator, kernel heap, and virtual memory subsystem properly
- **Load device drivers** — for disk controllers, network cards, etc., so the kernel can actually access the real root filesystem.
- **Mount the real root filesystem**
- **Start the init process** — the kernel creates **process 1**

- Next class(Coding tutorial): We will do OS Library calls
- Theory Class: Process management: Process (Remzi Ch4,Galvin Ch 3)